Swift Vapor - Cheat Sheet

Vapor 4.122.2 - Swift 6.4 - Fluent 4.13 - Leaf 4.5 - JWT 5.1
checked 2026-10-17 - albertoirurueta.github.io/docs - backend/vapor

THE STACK & A REQUEST'S LIFE

SwiftNIO (event loops, HTTP/1.1 + HTTP/2, TLS) -> **Vapor** (`Application`, router, `Request`/`Response`, `Content`) -> **Fluent**, **Leaf**, **Queues**, **JWT**, **Redis**. One process, async/await, Linux and macOS.

```
accept + decode -> middleware (first added = outermost)
 -> router match -> route-group middleware
 -> handler (async throws) -> ResponseEncodable
 -> encode -> middleware (back out) -> write
```

TOOLING & COMMAND MAP

```
swiftly install latest       # Swift 6.4 toolchain
brew install vapor           # Toolbox 20: `vapor new` only
vapor new Bookshelf          # then open Package.swift
swift build [-c release] | swift test
swift run App serve          # default cmd, :8080
  --hostname 0.0.0.0 --port 8080 --env production
swift run App routes         # print the route table
swift run App migrate [--revert] [--yes]
swift run App queues [--queue q | --scheduled]
swift run App seed           # your own AsyncCommand
```

Env: `--env`/`-e`, then `VAPOR_ENV`, else development. Logs: `--log`, `LOG_LEVEL`. `.env` and `.env.production` are loaded; `Environment.get("X")`, `Environment.secret(key:)` for file secrets.

PACKAGE.SWIFT

```
// swift-tools-version:6.3
let package = Package(name: "Bookshelf",
 platforms: [.macOS(.v13)],
 dependencies: [
  .package(url: "https://github.com/vapor/vapor.git",
           from: "4.122.2"),
  .package(url: "https://github.com/vapor/fluent.git",
           from: "4.13.0"),
  .package(url: "https://github.com/vapor/" +
           "fluent-postgres-driver.git", from: "2.14.0"),
  .package(url: "https://github.com/vapor/leaf.git",
           from: "4.5.2")],
 targets: [
  .executableTarget(name: "App", dependencies: [
   .product(name: "Vapor", package: "vapor"),
   .product(name: "Fluent", package: "fluent"),
   .product(name: "FluentPostgresDriver",
            package: "fluent-postgres-driver"),
   .product(name: "Leaf", package: "leaf")]),
  .testTarget(name: "AppTests", dependencies: ["App",
   .product(name: "VaporTesting", package: "vapor")])])
```

ENTRYPOINT.SWIFT & CONFIGURE.SWIFT

```
@main enum Entrypoint {
 static func main() async throws {
  var env = try Environment.detect()
  try LoggingSystem.bootstrap(from: &env)
  let app = try await Application.make(env)
  do { try await configure(app)
       try await app.execute()      // serve by default
  } catch { app.logger.report(error: error)
       try? await app.asyncShutdown(); throw error }
  try await app.asyncShutdown() } }

func configure(_ app: Application) async throws {
 app.middleware.use(FileMiddleware(publicDirectory:
   app.directory.publicDirectory))
 app.databases.use(.postgres(url: url), as: .psql)
 app.migrations.add(CreateBook())
 app.views.use(.leaf)
 try routes(app) }
```

SWIFT 6 CONCURRENCY RULES

Handlers are `@Sendable async throws` closures; new code uses async/await, not `EventLoopFuture`.

Fluent models are `final class ... : Model, @unchecked Sendable` (mutable property wrappers).

Bridging: `try await future.get()`; never block an event loop (`sleep`, sync file I/O, `Data(contentsOf:)`, heavy CPU) -- use `req.fileio`, a thread pool or a job.

Shared state: an `actor` or a `Sendable` struct stored with `app.storage` / `StorageKey`; no mutable globals.

Services: extensions on `Application` and `Request` (`req.client`, `req.db`, `req.view`); lifecycle via `LifecycleHandler` (`willBoot`, `didBoot`, `shutdown`).

DOCKERFILE STAGES & DEPLOYMENT CHECKLIST

```
FROM swift:6.3-noble AS build    # stage 1: builder
COPY ./Package.* ./              # cached layer
RUN swift package resolve
COPY . .
RUN swift build -c release --product App \
    --static-swift-stdlib -Xlinker -ljemalloc
                                 # copy binary -> /staging
FROM ubuntu:noble                # stage 2: runtime
                  # libjemalloc2 ca-certificates tzdata
COPY --from=build /staging /app  # + Public, Resources
USER vapor:vapor ; EXPOSE 8080
ENTRYPOINT ["./App"]
CMD ["serve", "--env", "production",
     "--hostname", "0.0.0.0", "--port", "8080"]
```

[ ] release build, `--env production` -- [ ] bind `0.0.0.0`, health route -- [ ] `migrate --yes` as a one-off job -- [ ] one scheduler replica -- [ ] secrets from env, no `.env` in the image -- [ ] non-root user -- [ ] SIGTERM -> graceful shutdown -- [ ] logs to stdout, metrics, traces.

ROUTING & CONTROLLERS

```
app.get("books") { req async throws -> [Book] in ... }
app.post("books", use: create)    // put patch delete
app.get("books", ":id") { req in
  try req.parameters.require("id") }  // String
app.get("files", "*")               // any one segment
app.get("docs", "**")               // the rest of the path
app.on(.POST, "up", body: .stream)  // .collect(maxSize:
                                    // "10mb") buffers up to a limit
let api = app.grouped("api", "v1")
let priv = api.grouped(User.authenticator(),
                       User.guardMiddleware())
struct BookController: RouteCollection {
 func boot(routes: any RoutesBuilder) throws {
  let b = routes.grouped("books")
  b.get(use: index); b.post(use: create)
  b.group(":id") { $0.get(use: show) } } }
try app.register(collection: BookController())
```

Default body limit is 16 KB (`app.routes.defaultMaxBodySize`). `Content` types are returned as JSON; `HTTPStatus` for bare codes.

REQUEST, RESPONSE, CONTENT, VALIDATION

```
struct NewBook: Content, Validatable {
 var title: String; var year: Int
 static func validations(_ v: inout Validations) {
  v.add("title", as: String.self, is: !.empty)
  v.add("year", as: Int.self, is: .range(1450...2100)) } }
try NewBook.validate(content: req)    // 400 + reasons
let b = try req.content.decode(NewBook.self)
let f = try req.query.decode(Filter.self)   // ?year=2020
req.headers.bearerAuthorization?.token
return Response(status: .created,
                body: .init(string: "ok"))
ContentConfiguration.global.use(encoder: enc, for: .json)
```

ERRORS & MIDDLEWARE

```
throw Abort(.notFound, reason: "no such book") // 404
struct AppError: AbortError {                  // custom
  var status: HTTPResponseStatus; var reason: String }
struct Timing: AsyncMiddleware {
 func respond(to req: Request, chainingTo next: any
   AsyncResponder) async throws -> Response {
  let r = try await next.respond(to: req); return r } }
app.middleware.use(Timing(), at: .beginning)
```

`ErrorMiddleware` (turns thrown errors into responses) and `RouteLoggingMiddleware` are installed by default. First added is outermost; group middleware wraps only matched routes (a 404 skips it). Built-ins: `FileMiddleware`, `CORSMiddleware`, `SessionsMiddleware`, `ResponseCompressionMiddleware`, `TracingMiddleware`.

FLUENT: WRAPPERS, MIGRATIONS, QUERIES

Drivers `fluent-postgres-driver` 2.14, `-sqlite-driver`, `-mysql-driver`, `-mongo-driver`. Wrappers: `@ID(key: .id)` `@Field(key:)` `@OptionalField` `@Parent(key:)` `@OptionalParent` `@Children(for: \.$a)` `@OptionalChild` `@Siblings(through:from:to:)` `@Timestamp(key:on:)` `@Enum(key:)` `@Group(key:)`

```
final class Book: Model, Content, @unchecked Sendable {
 static let schema = "books"
 @ID(key: .id) var id: UUID?
 @Field(key: "title") var title: String
 @Parent(key: "author_id") var author: Author
 @Timestamp(key: "created_at", on: .create)
 var createdAt: Date?
 init() {} }
struct CreateBook: AsyncMigration {
 func prepare(on db: any Database) async throws {
  try await db.schema("books").id()
   .field("title", .string, .required)
   .field("author_id", .uuid, .required,
          .references("authors", "id"))
   .unique(on: "title").create() }
 func revert(on db: any Database) async throws {
  try await db.schema("books").delete() } }
try await Book.query(on: req.db).filter(\.$year > 2000)
  .sort(\.$title).with(\.$author).range(..<20).all()
Book.find(id, on: db); .first(); .count(); .delete()
try await book.save(on: db)          // create / update
.paginate(for: req)   // ?page=1&per=20 -> Page<Book>
try await req.db.transaction { tx in
  try await a.save(on: tx); try await b.save(on: tx) }
if let sql = req.db as? any SQLDatabase {   // raw SQL
 try await sql.raw(
  "SELECT * FROM books WHERE year > \(bind: y)").all() }
```

SERVER, CONFIGURATION & SERVICES

```
app.http.server.configuration.hostname = "0.0.0.0"
app.http.server.configuration.port = 8080
app.routes.defaultMaxBodySize = "1mb"     // 16 KB default
struct SettingsKey: StorageKey {
 typealias Value = Settings }
extension Application { var settings: Settings {
  get { storage[SettingsKey.self]! }
  set { storage[SettingsKey.self] = newValue } } }
```

TLS in the process (`tlsConfiguration`, HTTP/2 needs TLS) or at the reverse proxy; read secrets from the environment at boot and fail fast.

STATIC FILES, API DESIGN, MONGODB

Static files: `FileMiddleware` serves `Public/` with ETag, `Cache-Control` and ranges; `req.fileio.streamFile(at:)` streams a file without blocking; never join user input into a path.

API design: return DTOs, not models; `/api/v1` prefix; `201` + `Location` on create, `204` on delete, `409` on a duplicate, `400` on validation errors; `PATCH` for partial updates; `Page` for lists.

MongoDB: `fluent-mongo-driver` 1.4 (`.mongo(connectionString:)`), `@ID(custom: .id)` with `ObjectId`; no schema migrations needed beyond indexes.

LEAF, SESSIONS, REDIS

```
#extend("base"):                        // layout uses #import
 #export("title", title)
 #export("body"):
  #for(book in books):<li>#(book.title)</li>#endfor
  #if(count(books) == 0): none #else: some #endif
 #endexport
#endextend
return try await req.view.render("home", ["title": "Hi"])
```

Tags: `#(x)` `#if` `#for` `#count` `#extend`/`#export`/`#import` `#date` `#uppercased` `#unsafeHTML` (raw) `#comment`. Output is HTML-escaped.

```
app.middleware.use(app.sessions.middleware)
app.sessions.use(.fluent)   // .memory .fluent .redis
req.session.data["uid"] = "1"; req.session.destroy()
app.redis.configuration = try RedisConfiguration(
  hostname: "localhost")
app.caches.use(.redis)          // .memory | .fluent
try await req.cache.set("k", to: v,
                        expiresIn: .minutes(5))
let v = try await req.cache.get("k", as: Book.self)
```

AUTH CHOOSER, JWT, PASSWORDS

Basic `AsyncBasicAuthenticator` / `ModelAuthenticatable` -> login endpoint only. **Bearer token** `AsyncBearerAuthenticator` / `ModelTokenAuthenticatable` -> API clients (store token hashes). **Session** `ModelSessionAuthenticatable` + sessions middleware -> browser apps. **JWT** `JWTPayload` -> stateless, microservices. **Sign in with Apple** -> verify the identity token.

```
let priv = app.grouped(User.authenticator(),
                       User.guardMiddleware())  // 401
let user = try req.auth.require(User.self)
req.auth.login(user)                            // session
let hash = try await req.password.async.hash("pw")
try await req.password.async.verify("pw", created: hash)
struct Claims: JWTPayload { var sub: SubjectClaim
 var exp: ExpirationClaim
 func verify(using alg: some JWTAlgorithm) async throws {
  try exp.verifyNotExpired() } }
await app.jwt.keys.add(hmac: HMACKey(from: secret),
  digestAlgorithm: .sha256, kid: "k1")
let t = try await req.jwt.sign(claims)
let c = try await req.jwt.verify(as: Claims.self)
try await req.jwt.apple.verify(token,
                               applicationIdentifier: id)
```

Security checklist: CSRF is not built in (session-bound token, `Origin`/`Sec-Fetch-Site` checks, `SameSite` cookies) -- headers (CSP, HSTS, nosniff) -- rate limits -- CORS allow-list, never `*` with credentials -- body size caps -- ownership checks (401 / 403 / 404) -- TLS at the proxy -- secrets from the environment -- short-lived access + rotating refresh tokens.

REAL TIME, UPLOADS, BACKGROUND WORK

```
app.webSocket("echo") { req, ws in
 ws.onText { ws, text in ws.send(text) }
 ws.onClose.whenComplete { _ in } }     // SSE: set
Response(headers: h,     // content-type text/event-stream
 body: .init(managedAsyncStream: { w in try await
  w.write(.buffer(ByteBuffer(string: "data: hi\n\n"))) }))
struct Up: Content { var file: File }    // multipart
try await req.fileio.writeFile(up.file.data, at: path)
struct Mail: AsyncJob { typealias Payload = MailMsg
 func dequeue(_ c: QueueContext,
              _ p: MailMsg) async throws {}
 func error(_ c: QueueContext, _ e: any Error,
            _ p: MailMsg) async throws {} }
try app.queues.use(.redis(url: url))
app.queues.add(Mail())
try await req.queue.dispatch(Mail.self, msg,
                             maxRetryCount: 3)
app.queues.schedule(Nightly()).daily().at(2, 0)
try app.queues.startScheduledJobs()
// or: swift run App queues --scheduled
let r = try await req.client.get("https://api.example/x")
let d = try r.content.decode(Dto.self)
```

APNs: `await app.apns.configure(.jwt(privateKey:, keyIdentifier:, teamIdentifier:))` (.p8 key), then `req.apns.client.sendAlertNotification(n, deviceToken:)`; delete tokens on 410. **Email:** no built-in client; provider-neutral `EmailSender` over SendGrid, Mailgun or SMTP, sent from a queued job.

OPENAPI, TESTING, OBSERVABILITY

OpenAPI: write `openapi.yaml` + `openapi-generator-config.yaml` -> `OpenAPIGenerator` plugin generates `APIProtocol` -> implement it -> `registerHandlers(on: VaporTransport(routesBuilder: app))`; client generated from the same file.

```
@Test func health() async throws {
 try await withApp(configure: configure) { app in
  try await app.testing().test(.GET, "health",
   afterResponse: { res async in
    #expect(res.status == .ok) }) } }
// .testing env, SQLite .memory, app.autoMigrate();
// app.testing(method: .running) = real server
req.logger.info("hit", metadata: ["id": "\(id)"])
app.middleware.use(TracingMiddleware())
let r = try await withSpan("db") { _ in try await load() }
Counter(label: "orders").increment()
MetricsSystem.bootstrap(PrometheusMetricsFactory(
  registry: registry))   // or swift-otel: OTel.bootstrap
```

CHANGED SINCE OLDER TUTORIALS

`Droplet`, `Services`, `config.prefer` -> `Application` and extensions (V4); `router.get` + `parameters.next` -> `app.get` + `parameters.get`

`EventLoopFuture` chains -> async/await; `main.swift` + `app.run()` -> `@main` entrypoint, `Application.make`, `execute()`

Leaf `#embed`/`#set` -> `#extend`/`#export`/`#import`; `vapor/auth` package -> built-in authenticators; `XCTVapor` -> `VaporTesting` + Swift Testing

Vapor Cloud, Heroku free tier -> Docker, Fly.io or Kubernetes. **Vapor 5 (beta.4, not GA):** swift-http-server, `Request`/`Response` structs, streamed bodies, macros for routes and middleware, swift-configuration